

ANSI SQL SOLUTION CODE

-- 1. USER UPCOMING EVENTS

-- Upcoming events, in the user's own city, that a given user is registered for.

-- Replace ? with an actual user_id (e.g. 1).

```
SELECT e.event_id, e.title, e.start_date, e.end_date
```

```
FROM Events e
```

```
JOIN Registrations r ON r.event_id = e.event_id
```

```
JOIN Users u ON u.user_id = r.user_id
```

```
WHERE r.user_id = ?
```

```
    AND e.city = u.city
```

```
    AND e.status = 'upcoming'
```

```
ORDER BY e.start_date;
```

-- 2. TOP RATED EVENTS

-- Only events with 10+ feedback entries qualify; highest average rating first.

```
SELECT e.event_id, e.title,
```

```
       AVG(f.rating) AS avg_rating,
```

```
       COUNT(*) AS feedback_count
```

```
FROM Events e
```

```
JOIN Feedback f ON f.event_id = e.event_id
```

```
GROUP BY e.event_id, e.title
```

```
HAVING COUNT(*) >= 10
```

```
ORDER BY avg_rating DESC;
```

```
-- 3. INACTIVE USERS
```

```
-- Users with no registration in the last 90 days.
```

```
SELECT u.*
```

```
FROM Users u
```

```
WHERE NOT EXISTS (
```

```
    SELECT 1 FROM Registrations r
```

```
    WHERE r.user_id = u.user_id
```

```
        AND r.registration_date >= CURDATE() - INTERVAL 90 DAY
```

```
);
```

```
-- 4. PEAK SESSION HOURS
```

```
-- Sessions starting between 10:00 and 12:00, counted per event.
```

```
SELECT event_id, COUNT(*) AS session_count
```

```
FROM Sessions
```

```
WHERE TIME(start_time) >= '10:00:00'
```

```
    AND TIME(start_time) < '12:00:00'
```

```
GROUP BY event_id;
```

```
-- 5. MOST ACTIVE CITIES
```

```
-- Top 5 cities by number of distinct users who registered for events.
```

```
SELECT u.city, COUNT(DISTINCT r.user_id) AS distinct_users
```

```
FROM Registrations r
JOIN Users u ON u.user_id = r.user_id
GROUP BY u.city
ORDER BY distinct_users DESC
LIMIT 5;
```

-- 6. EVENT RESOURCE SUMMARY

-- Breakdown of resource types uploaded per event (0 if none).

```
SELECT e.event_id, e.title,
       SUM(res.resource_type = 'pdf') AS pdf_count,
       SUM(res.resource_type = 'image') AS image_count,
       SUM(res.resource_type = 'link') AS link_count,
       COUNT(res.resource_id) AS total_resources
FROM Events e
LEFT JOIN Resources res ON res.event_id = e.event_id
GROUP BY e.event_id, e.title;
```

-- 7. LOW FEEDBACK ALERTS

-- Feedback with rating below 3, with the reviewer and event name attached.

```
SELECT u.full_name, e.title AS event_name, f.rating, f.comments
FROM Feedback f
JOIN Users u ON u.user_id = f.user_id
JOIN Events e ON e.event_id = f.event_id
WHERE f.rating < 3;
```

-- 8. SESSIONS PER UPCOMING EVENT

```
SELECT e.event_id, e.title, COUNT(s.session_id) AS session_count
FROM Events e
LEFT JOIN Sessions s ON s.event_id = e.event_id
WHERE e.status = 'upcoming'
GROUP BY e.event_id, e.title;
```

-- 9. ORGANIZER EVENT SUMMARY

-- Number of events each organizer created, broken down by status.

```
SELECT u.user_id, u.full_name, e.status, COUNT(*) AS event_count
FROM Events e
JOIN Users u ON u.user_id = e.organizer_id
GROUP BY u.user_id, u.full_name, e.status;
```

-- 10. FEEDBACK GAP

-- Events that have registrations but zero feedback submissions.

```
SELECT DISTINCT e.event_id, e.title
FROM Events e
JOIN Registrations r ON r.event_id = e.event_id
LEFT JOIN Feedback f ON f.event_id = e.event_id
WHERE f.feedback_id IS NULL;
```

-- 11. DAILY NEW USER COUNT

-- New signups per day, last 7 days.

```
SELECT registration_date, COUNT(*) AS new_users
FROM Users
WHERE registration_date >= CURDATE() - INTERVAL 7 DAY
GROUP BY registration_date
ORDER BY registration_date;
```

-- 12. EVENT WITH MAXIMUM SESSIONS

-- Handles ties: returns every event that shares the top session count.

```
WITH session_counts AS (
    SELECT event_id, COUNT(*) AS session_count
    FROM Sessions
    GROUP BY event_id
)
SELECT sc.event_id, e.title, sc.session_count
FROM session_counts sc
JOIN Events e ON e.event_id = sc.event_id
WHERE sc.session_count = (SELECT MAX(session_count) FROM session_counts);
```

-- 13. AVERAGE RATING PER CITY

```
SELECT e.city, AVG(f.rating) AS avg_rating
FROM Events e
```

```
JOIN Feedback f ON f.event_id = e.event_id
```

```
GROUP BY e.city;
```

```
-- 14. MOST REGISTERED EVENTS
```

```
-- Top 3 events by total registration count.
```

```
SELECT e.event_id, e.title, COUNT(r.registration_id) AS total_registrations
```

```
FROM Events e
```

```
JOIN Registrations r ON r.event_id = e.event_id
```

```
GROUP BY e.event_id, e.title
```

```
ORDER BY total_registrations DESC
```

```
LIMIT 3;
```

```
-- 15. EVENT SESSION TIME CONFLICT
```

```
-- Self-join to find any two sessions in the same event whose time ranges overlap.
```

```
SELECT a.event_id,
```

```
       a.session_id AS session_1, a.start_time AS s1_start, a.end_time AS s1_end,
```

```
       b.session_id AS session_2, b.start_time AS s2_start, b.end_time AS s2_end
```

```
FROM Sessions a
```

```
JOIN Sessions b
```

```
  ON a.event_id = b.event_id
```

```
  AND a.session_id < b.session_id    -- avoid duplicate/mirrored pairs
```

```
  AND a.start_time < b.end_time
```

```
  AND b.start_time < a.end_time;    -- classic overlap condition
```

-- 16. UNREGISTERED ACTIVE USERS

-- Signed up in the last 30 days but never registered for an event.

```
SELECT u.*  
FROM Users u  
LEFT JOIN Registrations r ON r.user_id = u.user_id  
WHERE u.registration_date >= CURDATE() - INTERVAL 30 DAY  
AND r.registration_id IS NULL;
```

-- 17. MULTI-SESSION SPEAKERS

-- Speakers handling more than one session, across all events.

```
SELECT speaker_name, COUNT(*) AS session_count  
FROM Sessions  
GROUP BY speaker_name  
HAVING COUNT(*) > 1;
```

-- 18. RESOURCE AVAILABILITY CHECK

-- Events with zero uploaded resources.

```
SELECT e.event_id, e.title  
FROM Events e  
LEFT JOIN Resources res ON res.event_id = e.event_id  
WHERE res.resource_id IS NULL;
```

-- 19. COMPLETED EVENTS WITH FEEDBACK SUMMARY

```
SELECT e.event_id, e.title,
       COUNT(DISTINCT r.registration_id) AS total_registrations,
       AVG(f.rating) AS avg_rating
FROM Events e
LEFT JOIN Registrations r ON r.event_id = e.event_id
LEFT JOIN Feedback f ON f.event_id = e.event_id
WHERE e.status = 'completed'
GROUP BY e.event_id, e.title;
```

-- 20. USER ENGAGEMENT INDEX

-- Events attended (distinct) and feedback given, per user.

```
SELECT u.user_id, u.full_name,
       COUNT(DISTINCT r.event_id) AS events_attended,
       COUNT(DISTINCT f.feedback_id) AS feedback_submitted
FROM Users u
LEFT JOIN Registrations r ON r.user_id = u.user_id
LEFT JOIN Feedback f ON f.user_id = u.user_id
GROUP BY u.user_id, u.full_name;
```

-- 21. TOP FEEDBACK PROVIDERS

-- Top 5 users by number of feedback entries submitted.

```
SELECT u.user_id, u.full_name, COUNT(f.feedback_id) AS feedback_count
FROM Users u
```



```
JOIN Feedback f ON f.user_id = u.user_id

GROUP BY u.user_id, u.full_name

ORDER BY feedback_count DESC

LIMIT 5;
```

-- 22. DUPLICATE REGISTRATIONS CHECK

-- Same user registered for the same event more than once.

```
SELECT user_id, event_id, COUNT(*) AS times_registered

FROM Registrations

GROUP BY user_id, event_id

HAVING COUNT(*) > 1;
```

-- 23. REGISTRATION TRENDS

-- Month-wise registration counts over the past 12 months.

```
SELECT DATE_FORMAT(registration_date, '%Y-%m') AS reg_month,

       COUNT(*) AS registration_count

FROM Registrations

WHERE registration_date >= CURDATE() - INTERVAL 12 MONTH

GROUP BY reg_month

ORDER BY reg_month;
```

-- 24. AVERAGE SESSION DURATION PER EVENT

-- Duration in minutes, averaged across all sessions in an event.

```
SELECT event_id, AVG(TIMESTAMPDIFF(MINUTE, start_time, end_time)) AS  
avg_duration_minutes  
FROM Sessions  
GROUP BY event_id;
```

-- 25. EVENTS WITHOUT SESSIONS

```
SELECT e.event_id, e.title  
FROM Events e  
LEFT JOIN Sessions s ON s.event_id = e.event_id  
WHERE s.session_id IS NULL;
```

GUNASHREE V

111623102066

RMKCET